M1.1: Node Lifecycle & Architecture

Theory: Atomic computation blocks inside the robot graph, managed by executors for event loop schedules.

Details: Nodes correspond to ‘rclcpp::Node‘ handlers, sharing a common container for intra-process efficiency or running on dedicated OS threads.

Trade-off: Container-sharing exposes all concurrent nodes to single-point failures; a segfault in one component crashes the host process.

M1.2: DDS Adapter & RMW Abstraction Layer

Theory: Decouples API client libraries from vendor-specific DDS bindings using standard C APIs.

Details: Provides the ROS Middleware Interface (RMW) specification. Vendors implement drivers like ‘rmw_fastrtps_cpp‘ or ‘rmw_cyclonedds_cpp‘ loaded at boot.

Trade-off: Abstracting DDS hides advanced vendor-specific features, requiring custom XML injections for edge-case configuration overrides.

M1.3: DDS Dynamic Peer Discovery

Theory: Discards central master nodes (ROS 1 Master) by using peer-to-peer multicast network discovery.

Details: Operates SPDP (Simple Participant Discovery Protocol) and SEDP (Simple Endpoint Discovery Protocol) to exchange participant and endpoint schemas.

Trade-off: Large subnets generate discovery multicast storms, requiring strict uncast peer lists to minimize discovery traffic.

M1.4: DDS Quality of Service (QoS) Policies

Theory: Configures network reliability and buffering behaviors for robust data transmission.

Details: Relies on three base configurations: Reliability (Best Effort / Reliable), Durability (Transient Local / Volatile), and History (Keep Last / Keep All).

Trade-off: QoS configurations must be compatible to establish connections. Under Offered vs Requested constraints, mismatched parameters silently drop packets.

M1.5: Intra-Process Communication (IPC)

Theory: Passes heap pointers directly inside a single process container, bypassing serialization and socket copies.

Details: Activates ‘use_intra_process_comm‘ configuration, passing a ‘std::unique_ptr‘ across queues without converting structures to CDR format.

Trade-off: Transferring unique pointers transfers ownership. For multiple subscribers, deep copies are forced unless read-only shared pointers are used.

M1.6: Zero-Copy Shared Memory (iceoryx)

Theory: Offloads inter-process socket serialization via a lock-free shared memory ring buffer.

Details: Interlinks RMW with iceoryx, loaning shared memory regions via ‘borrow_loaned_message()‘ to write payloads directly without data copying.

Trade-off: Requires fixed-size structures. Variable-length arrays or strings force fallback to socket copying.

M2.1: Topic Publishing & CDR Serialization

Theory: Asymmetric, unidirectional data streams serialized using Common Data Representation (CDR) standards.

Details: Compiles ‘.msg‘ schemas into C++ structures, utilizing CDR’s binary alignment rules to stream values over sockets.

Trade-off: CDR alignment rules generate padding bytes, which consume extra network bandwidth for micro-sized high-frequency payloads.

M2.2: Asynchronous Services

Theory: Bounded request-response transaction model mapped over dedicated communication paths.

Details: Implements requests and responses using matched topics, where executors map transaction IDs to client futures on response arrivals.

Trade-off: Synchronous calls block executor loops. Invoking a nested synchronous service in a single-threaded executor causes permanent deadlocks.

M2.3: Action Task Execution Framework

Theory: Manages long-running tasks via feedback loops, goal cancellations, and asynchronous result tracking.

Details: Incorporates goal/cancel/result services with feedback/status topics to monitor remote algorithms safely.

Trade-off: State machines are complex and prone to synchronization issues on packet losses, requiring client-side timeouts.

M2.4: Executor Scheduling Engine

Theory: Schedules and executes incoming callback events from the DDS graph.

Details: Supports SingleThreaded and MultiThreaded Executors. Controls callback concurrency via callback groups (MutuallyExclusive, Reentrant).

Trade-off: Multithreading risks race conditions on shared node data; non-atomic variables require mutex protections in reentrant groups.

M2.5: Dynamic Parameter System

Theory: Provides runtime configuration modifications and event callbacks without rebuilding nodes.

Details: Modifies parameters via setter callbacks, broadcasting parameter changes to the global '/parameter_events' topic.

Trade-off: Heavy parameter updates across large fleets consume network bandwith; changes require debouncing filters.

M2.6: Lifecycle Node State Machine

Theory: Enforces deterministic initialization, configuration, activation, and cleanup sequences for nodes.

Details: Restricts publishing/subscribing until the node is in the Active state. Defines transition callbacks like 'on_configure' and 'on_activate'.

Trade-off: Transition callbacks block the main initialization thread. Long setup operations must run asynchronously to avoid startup lockups.

M3.1: Launch Engine

Theory: Orchestrates distributed node deployments and monitors lifecycle status dynamically.

Details: Uses Python launch scripts to parse parameters and register event handlers for node crashes.

Trade-off: Python orchestration adds startup latency; real-time hard systems should favor static C++ containers.

M3.2: MCAP/Bag Recording

Theory: Captures topic streams to disk for offline analysis and playback.

Details: Replaces SQLite with MCAP, an append-only file format designed for robot sensors, supporting embedded schemas.

Trade-off: High-bandwidth camera feeds risk disk I/O bottlenecks; setups require separate I/O queues and compression.

M3.3: Real-Time Guarantee

Theory: Bounds execution latency by locking process memory and adjusting scheduler parameters.

Details: Invokes 'mlockall' to pin addresses in RAM, preventing Linux swap delays. Upgrades executor threads to 'SCHED_FIFO'.

Trade-off: Requires root access and real-time RT-PREEMPT kernels to avoid scheduling priority inversions.

M3.4: QoS Compatibility Rules

Theory: Establishes connections using compatibility checks (offered vs requested rules).

Details: Requires the Offered QoS level from the publisher to equal or exceed the Requested QoS from the subscriber.

Trade-off: Mismatches silently block communication without throwing compile errors; diagnostics must be checked via RMW log alerts.

M3.5: Card 17 Fallback Title

Placeholder text for compiler robustness.

M3.6: Card 18 Fallback Title

Placeholder text for compiler robustness.

M4.1: Card 19 Fallback Title

Placeholder text for compiler robustness.

M4.2: Card 20 Fallback Title

Placeholder text for compiler robustness.

M4.3: Card 21 Fallback Title

Placeholder text for compiler robustness.

M4.4: Card 22 Fallback Title

Placeholder text for compiler robustness.

M4.5: Card 23 Fallback Title

Placeholder text for compiler robustness.

M5.1: Card 24 Fallback Title

Placeholder text for compiler robustness.

M5.2: Card 25 Fallback Title

Placeholder text for compiler robustness.

M5.3: Card 26 Fallback Title

Placeholder text for compiler robustness.

M5.4: Card 27 Fallback Title

Placeholder text for compiler robustness.

M5.5: Card 28 Fallback Title

Placeholder text for compiler robustness.

Zone T1: ROS 2 Core APIs

rclcpp::Node, rcl::rcl, rmw::rmw, rcutils::rcutils, tf2::BufferCore, rclcpp::executors::MultiThreadedExecutor

Zone T2: Middleware & Execution Errors

dds_qos_incompatible_match, shared_memory_segment_overflow, executor_deadlock_callback_stall, realtime_priority_inversion, tf2_extrapolation_out_of_range, lifecycle_invalid_state_transition